

Unitat 8.1

PL/SQL: Estructures condicionals i iteratives

Estructures condicionals

2 estructures condicionals:

- IF: Avalua una o més condicions i executa el codi que correspongui segons la certesa d'aquestes.
- CASE: Avalua una expressió i, depenent del seu valor, executa una porció de codi o una altra.

IF

Estructura bàsica (alternativa simple):

```
IF <condició> THEN  
    <sentències_si_condició_certa>;  
END IF;
```

IF

ALTERNATIVA DOBLE

```
IF <condició> THEN
    <sentències_si_cert>;
ELSE
    <sentències_si_fals>;
END IF;
```

ALTERNATIVA MÚLTIPLE

```
IF <condició1> THEN
    <sentències_si_c1_cert>;
ELSIF <condició2> THEN
    <sentències_si_c2_cert>;
ELSIF ...
ELSE
    <sentències_si_cN_fals>;
END IF;
```

IF: exemple



```
DECLARE
    ID_EMPLEAT NUMBER := 100;
    SALARI NUMBER;
BEGIN
    SELECT SALARY INTO SALARI FROM HR.EMPLOYEES WHERE EMPLOYEE_ID = ID_EMPLEAT;
    IF SALARI > 20000 THEN
        DBMS_OUTPUT.PUT_LINE('Cobra més de 20000 euros');
    ELSIF SALARI < 20000 THEN
        DBMS_OUTPUT.PUT_LINE('Cobra menys de 20000 euros');
    ELSE
        DBMS_OUTPUT.PUT_LINE('Cobra 20000 euros');
    END IF
END;
```

CASE

Molt semblant al CASE de SQL.

2 tipus:

- CASE simple (*simple case*)
- CASE examinat (*searched case*)

CASE simple

Econdicional basada en la igualtat dels operands.

L'expressió de cerca s'avalua una vegada i es compara successivament amb cada expressió de les clàusules WHEN.

Estructura:

```
CASE expressio-de-cerca
    WHEN expressio [, expressio [ ... ]] THEN
        sentències
    [ WHEN expressio [, expressio [ ... ]] THEN
        sentències ... ]
    [ ELSE sentències ]
END CASE;
```

CASE simple: exemple

```
DECLARE
    ID_EMPLEAT NUMBER := 100;
    SALARI NUMBER;
BEGIN
    SELECT SALARY INTO SALARI FROM HR.EMPLOYEES WHERE EMPLOYEE_ID = ID_EMPLEAT;
    CASE SALARI
        WHEN 1000 THEN
            DBMS_OUTPUT.PUT_LINE('És mileurista');
        WHEN 2000 THEN
            DBMS_OUTPUT.PUT_LINE('És dosmileurista');
        ELSE
            DBMS_OUTPUT.PUT_LINE('No sé sap');
    END CASE;
END;
```


CASE examinat

Execució condicional basada en la veritat de les expressions booleanes. Cada clàusula WHEN inclou una expressió booleana que s'avalua al seu torn, fins que se'n troba una en què el resultat és veritat.

Estructura bàsica:

CASE

 WHEN expressió-booleana THEN
 sentències

 [WHEN expressió-booleana THEN sentències ...]

 [ELSE sentències]

END CASE;

CASE examinat: exemple

```
DECLARE
    ID_EMPLEAT NUMBER := 100;
    SALARI NUMBER;
BEGIN
    SELECT SALARY INTO SALARI FROM HR.EMPLOYEES WHERE EMPLOYEE_ID = ID_EMPLEAT;
    CASE
        WHEN salari > 20000 THEN
            DBMS_OUTPUT.PUT_LINE('Cobra més de 20000 euros');
        WHEN salari < 20000 THEN
            DBMS_OUTPUT.PUT_LINE('Cobra menys de 20000 euros');
        ELSE
            DBMS_OUTPUT.PUT_LINE('Cobra 20000 euros');
    END CASE;
END;
```

Exercici estructures condicionals

Fes un bloc anònim que, combinant IF i CASE:

1. Agafi el JOB_TITLE d'un empleat concret, l'ID del qual s'ha de passar a través d'una variable.
2. Si aquest JOB_TITLE no és 'Programmer', imprimeix en pantalla un missatge indicant que no és programador.
3. Si aquest JOB_TITLE és 'Programmer', agafa la data de contractació del treballador i comprova el seu valor. Imprimeix el trimestre de contractació segons aquesta data: 1 (De Gener a Març), 2 (D'Abril a Juny), 3 (De Juliol a Setembre), 4 (D'Octubre a Desembre).

Utilitza l'esquema HR (taula EMPLOYEES)

Estructures iteratives

Executen un conjunt d'instruccions repetidament.

3 tipus d'iteracions/bucles:

- LOOP
- WHILE ... LOOP
- FOR

LOOP

Executa un conjunt de sentències repetidament.

Estructura bàsica (*loop infinit*):

LOOP

<conjunt_de_sentències>

END LOOP;

EXIT WHEN

Especifica una condició de sortida. Quan es compleix, el LOOP s'atura.

LOOP

<conjunt_de_sentències>

EXIT WHEN <condició_sortida>

END LOOP;

LOOP: exemple



```
DECLARE
    i number(3,0) := 0;
BEGIN
    LOOP
        i := i + 1;
    END LOOP;
END;
```

i s'incrementa indefinidament



```
DECLARE
    i number(3,0) := 0;
BEGIN
    LOOP
        i := i + 1;
        EXIT WHEN i = 999;
    END LOOP;
END;
```

i s'incrementa fins 999

WHILE LOOP

Executa unes sentències mentre es compleixi una condició.

Estructura idèntica a la de la majoria de llenguatges:

```
WHILE <condició> LOOP  
    <conjunt_de_sentències>  
END LOOP;
```


WHILE LOOP: exemple



```
DECLARE
    i number(3,0) := 0;
BEGIN
    WHILE i < 999 LOOP
        i := i + 1;
    END LOOP;
END;
```

FOR LOOP

Permet repetir sentències un nombre concret de vegades.

Estructura:

```
FOR <índex> IN [REVERSE] <mínim>..<màxim> LOOP  
    <conjunt_de_sentències>  
END LOOP;
```

FOR LOOP

IMPORTANT!

- L'índex és una variable que no cal declarar, es crearà en començar el bucle i destruirà en acabar-lo. Si ja hi ha una variable amb aquest nom, quedarà oculta dins el bucle.
- El rang màxim (costat dret) ha de ser més gran que el rang mínim (costat esquerra). Si no, no s'executarà cap iteració.
- El mínim i el màxim de l'iteració poden ser variables.
- REVERSE (opcional) fa que l'índex vagi del màxim al mínim, decrementant a cada iteració.

FOR: example



```
DECLARE
```

```
    i number(3,0) := 0;
```

```
BEGIN
```

```
    FOR idx IN 1..999 LOOP
```

```
        i := i + 1;
```

```
    END LOOP;
```

```
END;
```

Sentències de control d'iteració

Es poden utilitzar a qualsevol bucle (LOOP, WHILE, FOR).

- EXIT: Surt del bucle. (Similar a *break*)
- CONTINUE: Passa a la següent iteració.

Exercici estructures iteratives

Sobre l'esquema "U6_Biblioteca.sql"

1. Crea una taula VENDA amb la següent informació: DNI client, ID llibre (FK a LLIBRE), data venda (TIMESTAMP) i preu.
2. Agafa un llibre qualsevol (el seu ID ha d'estar dins una variable) i itera un nombre de vegades igual al nombre d'exemplars del llibre. Per cada iteració:
 1. Redueix en 1 el nombre d'exemplars del llibre (a la taula, UPDATE).
 2. Inserta un registre a la taula VENDA amb el DNI d'un client predefinit (es pot guardar a una constant), l'ID del llibre, el timestamp actual (SYSTIMESTAMP) i un preu qualsevol.

Recorreguts sobre col·leccions

Es poden utilitzar les iteracions per recórrer col·leccions.

Important:

- Quan s'ha d'aturar el recorregut:
 - Quan no hi ha proper índex (associative array, nested table o varray)
 - Quan el núm de l'índex és igual a la mida de la col·lecció (varray).
- Com passar al següent element:
 - Utilitzar NEXT amb l'índex actual (associative array, nested table o varray).
 - Sumar 1 a l'índex actual (varray).

Example: Associative array



```
DECLARE
  TYPE arr_t IS TABLE OF VARCHAR2(30) INDEX BY VARCHAR2(30);
  l_arr arr_t;
  l_idx VARCHAR2(30);
BEGIN
  l_arr('a') := 'x';
  l_arr('b') := 'y';

  l_idx := l_arr.FIRST;
  WHILE l_idx IS NOT NULL LOOP
    dbms_output.put_line(l_idx || '=' || l_arr(l_idx));
    l_idx := l_arr.NEXT(l_idx);
  END LOOP;
END
```


Exemple: VARRAY

```
DECLARE
  TYPE t_varr IS VARRAY(5) OF LLIBRE.TITOL%TYPE;
  l_varr t_varr;
BEGIN
  SELECT titol BULK COLLECT INTO l_varr FROM LLIBRE ORDER BY exemplars DESC FETCH FIRST 5 ROWS
  ONLY;
  FOR i IN l_varr.FIRST..l_varr.LAST LOOP
    DBMS_OUTPUT.PUT_LINE(i||': '||l_varr(i));
  END LOOP;
END
```

Example: Nested table

```
DECLARE
  TYPE t_varr IS TABLE OF VARCHAR2(255);
  l_varr t_varr;
  l_idx number;
BEGIN
  SELECT ID_ASSIGNATURA || ' - ' || dni_alumne || ' - ' || nota BULK COLLECT INTO l_varr FROM NOTA
    ORDER BY nota DESC FETCH FIRST 5 ROWS ONLY;

  l_idx := l_varr.FIRST;
  WHILE l_idx IS NOT NULL LOOP
    dbms_output.put_line(l_idx || ': ' || l_varr(l_idx));
    l_idx := l_varr.NEXT(l_idx);
  END LOOP;
END;
```